天猫技术大型 AI 项目的研发协同实践

原创 天猫技术团队 大淘宝技术 2026年9月7日 05:09 贵州 67人

本文以天猫新品 MDI 项目为例，阐述了大型 AI 项目研发协同从“超级个体”到“全员赋能”的四个进化阶段：在单兵攻坚期，通过构建结构化知识库让 AI 深度理解项目上下文，打造“超级个体”；在加人提速期，借鉴站点地图思路进行物理隔离分工，利用 Spec 变更规则及上下文缝补机制（MCP）解决多人协作冲突与推理链断裂，实现效率横向复刻；在高速迭代期，部署独立“技术哨兵机”还原预发合并态，通过云端 Agent 检测隐形业务逻辑冲突并闭环告警，保障研发质量不劣化；在稳定运作期，推出 NekoCollar 云端协作平台，通过沙箱环境、所见即所得的真实页面代理及 Git 软硬双锁，让非研发角色也能安全、高效地基于代码上下文参与协同，最终实现 AI 研发能力从个人到团队再到组织全员的规模化放大。

写在前面：为什么这个项目值得写

AI 作为生产力工具，带来的不是一次从顶层发生的效率革命，而是一条从一线生长、向组织逐级放大的生产力曲线。只有使效率从“个人偶得”转变为“团队可复制”，AI Native 的成熟实践才有可能跨团队、跨业务复制，形成组织乃至集团层面的规模效应。

这几个月我们在做一个全新的业务项目——天猫新品 MDI 项目。我们在演进的过程就经历了上面所述的从 ***超级个体 -> 超级研发小队 -> 超“稳”研发小队 -> 超级项目组*** 的进化过程。

这个多位同学并行采用 SDD 规格驱动高频敏捷开发，严格每周线上迭代一个大版本。最夸张的一次上线的时候，同时存在八个仓库、十几个分支在预发集成开发。本文就要说一下我们的这随着项目组进化的四个过程里，遇到了什么坑，又做了些什么。

它不是一个单点功能，而是一个横跨 4 个角色端、约 40 个功能模块的全栈平台，再叠加一套外部智能投放系统、算法内核与自动化评测体系、阿里云百炼全模态等外部依赖，整体复杂程度非常大。

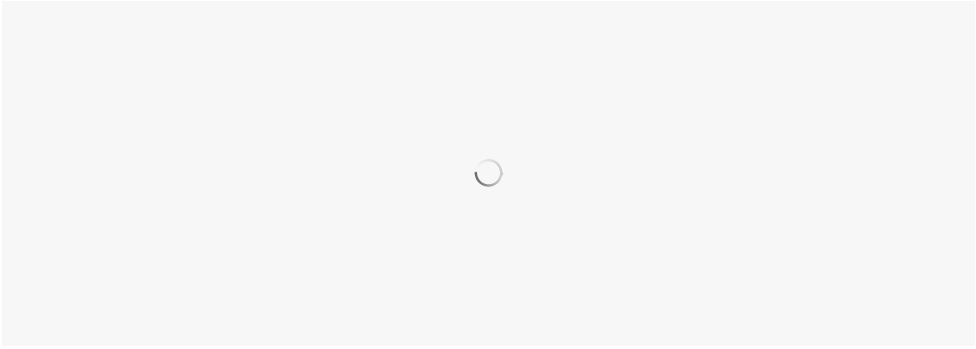
这个初创项目几乎完整命中了「AI 时代项目」的所有显著特点，它经历了四个非常典型的阶段：

- **单兵攻坚期**：单兵如何启动探索，攻坚新领域与新知识
- **加人提速期**：如何把单人效率复刻到多人协同上，逐步加速项目进程
- **高速迭代期**：如何消解时间与规模带来的 AI 研发质量劣化
- **稳定运作期**：项目组全部成员如何共享上下文，推进非研发角色的群策群力

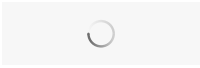
这四个阶段串起来，要解决的其实就是一条朴素的如何「能力放大」的问题：

- **第一步，让开发者成为“超级个体”。**
- **第二步，让超级个体组成“超级研发团队”。**
- **第三步，让这支团队可持续地跑下去。**
- **第四步，把研发能力辐射到项目组的每个非研发人员上。**

个体 → 团队 → 可持续 → 全员，一条不断往外放大的路。



AI 项目的四阶段



单兵攻坚期：构建极具纵深的「超级个体」编码上下文

问题：如何驱动 AI 在新领域、新知识上开垦？

项目启动时只有一两个人。做的是一个全新领域。单兵作战时，最大的痛点不是「不会写代码」，而是 **AI 不知道这个项目是什么**。它不知道我们的业务地图、不知道我们踩过哪些坑、不知道我们为什么选了这个技术方案。每开一个新会话，上下文都要重新灌一遍，效率极低。

我们的解题思路是：**用一个结构化的知识库，把整个全栈项目集成在一起，让 AI 随时能“读懂”这个项目**。这就是我们称之为 collar（项圈）的 SDD 研发体系。

> collar（项圈），给自由奔放的 AI Agent 套上一个边界与知识的项圈。这个命名也有渊源：我们有一系列以 neko（日语「猫」）命名的产品（比如 neko，比如 nekoClaw，还有后文解法四的 NekoCollar），而 collar 正是给「猫」戴的项圈/牵引具（harness）

手段一：AGENTS.md，任何时候都尤为重要的 AI 编码全栈地图

根目录的一份 `AGENTS.md` 。它是 Agent 进入这个项目的唯一入口。设计上我们给它定了一条铁律，就写在文件开头：

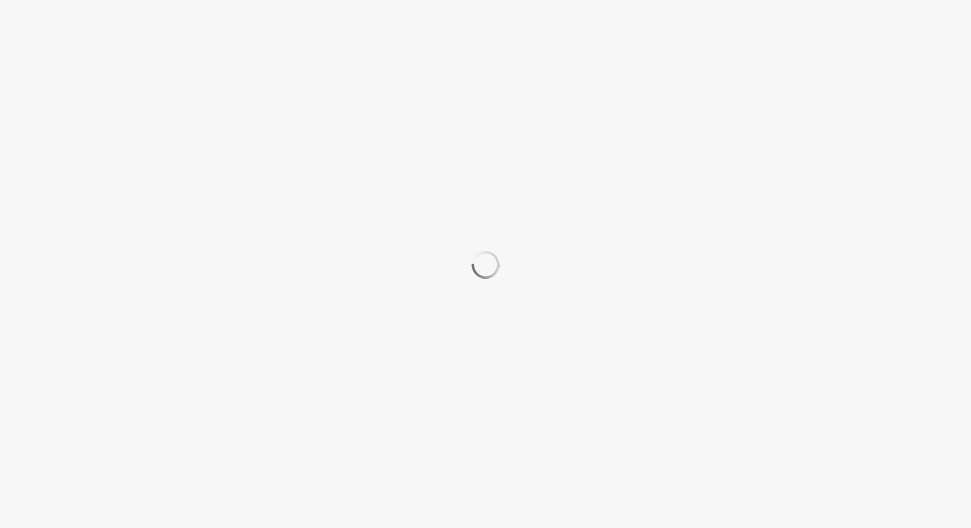
> 本文件是 Agent 的入口地图（≤120 行）。不放细节，只告诉你去哪里找。

我们 MDI 是一个多仓库聚合工作区（multi-repo workspace）：前端主应用、5 个微应用/H5、Java 后端（DDD 四层）、Node 评测执行端、评测服务端……全都作为独立 git 仓库聚合在同一个 `workspace-mdi/` 目录下。
AGENTS.md 用几个精炼的板块把这一切串起来：

- **仓库地图**：一棵带注释的目录树，一眼看清 7 个子仓库各自是什么、端口多少、哪个是“唯一开发入口”、哪个“已废弃禁止改动”。
- **快速命令**：每个应用怎么启动、怎么 build/lint，AI 不用去翻各仓 README。
- **关键约定速查表**：用一张表格，把项目里所有“踩过坑才总结出来的强约定”浓缩成「一句话 + 详见链接」的形式。例如：

```
| 约定 | 一句话 | 详见 |
|---|---|---|
| 时间字段展示 | 禁止裸渲染时间戳，统一格式化为北京时间 | CONVENTIONS.md |
| 日志体系 | 新增日志强制 BizLog，入口方法必加 @BLog | CONVENTIONS.md |
| MTOP 路由 | 新接口用 @MtopController + @MtopMethod | ADR-013 |
| 踩坑必读 | 改代码前先看 PITFALLS | PITFALLS.md |
```

- **当前状态表**：每个应用「完成度 + 已具备能力」的实时快照，让 AI 知道现在做到哪了。



AGENTS.md 入口地图 —— 每个会话必加载，所以它只导航、不堆细节

AGENTS.md 的行数由 `collar-runbook` skill 主动“守卫”：一旦逼近 115 行，skill 会提醒把详情迁到 `docs/`、AGENTS.md 只留一行摘要 + 链接，保证入口地图永远精简可信。

手段二：六个 Skill 全方位管理知识库的六个层级

光有约束还不够，我们还要解决「知识往哪写、什么时候自动写」的问题。我们把项目知识库拆成 docs/ 下的六个模块，每个模块配一个**唯一对应的 Skill**（统一 `collar-` 前缀命名），构成「模块 ↔ Skill ↔ README」三位一体：

```
docs/
├── specs/         业务地图（要做什么）          → collar-specs
├── changelog/     做了什么 + 将做什么（时间线）  → collar-changelog
├── architecture/  系统怎么连的（8 类连接点）     → collar-architecture
├── runbook/       零散过程知识（8 类）           → collar-runbook
├── vendor/        外部源码选择性同步             → collar-vendor
└── wiki/          自由知识库（人工维护，中文）    → collar-wiki
```

这六个 Skill 最关键的设计，是它们的**触发方式有主动和被动之分，自动化程度呈梯度**：

我们认为，**「对 Agent 来说，看不到的知识等于不存在。」**所以所有知识必须版本化落到仓库文件里，一旦发生就自动沉淀进 `docs/`，下一个会话或下一个人拉取仓库就能获得完整上下文。

单个 Skill 好做，难的是让多个 Skill 不打架、还能配合。我们的办法是把 **git commit 这个动作设计成统一的触发关卡**——一次提交，三个 Skill 在同一时刻各司其职、从三个正交维度同时喂养知识库：

- **collar-changelog 记「做了什么」**——从这次提交里提取变更，落到时间线。
- **collar-runbook 抽「学到了什么」**——把这轮踩的坑、定的约定沉淀成过程知识。
- **collar-specs 检「代码是否偏离了意图」**——比对代码和 spec，把差异摆出来让人决策。

而且这里的「强制」是真强制，这样知识沉淀就从「靠自觉」变成了「流程门禁」，杜绝了「这次先算了、下次一定」的滑坡。毕竟一旦开始欠账，知识库很快就会烂掉。

六个模块：本质上是知识“三”个方向上的互补的“两”面

手段三：人工与自动双方式实现知识库冷启

新领域的前期调研，最考验知识管理和冷启。我们的做法是把知识分成 **人工维护** 和 **自动维护** 两类，各归其位：

- **人工维护（需要个人保障质量）**：放进 `wiki/`（自由知识库，如设计理念、竞品分析、blue-print 蓝图调研）
- **自动维护**：`changelog / architecture / runbook` 三层由 AI 从 git、代码、对话中自动提取沉淀。

而对于**官方 demo 代码**这类外部参考资产，我们走 `vendor/` 模块，核心原则是「**只做减法、只下载我们所需要的**」。`collar-vendor` Skill 定义了两种同步方式和一套选择算法来做外部代码资产管理，更妙的是，vendor 不只是“存代码”，它还是 `collar-specs` 的**实证参考源**。当 specs 生成技术方案时，会自动扫描相关 vendor，提取代码模板、API 用法、架构模式，融入方案并标注来源。一句点睛的话（写在 skill 里）：

> 「用户放进 vendor 的代码不一定完整阅读过，这一步帮用户『读』并提炼要点。」

人在调研期把粗糙素材随手丢进 vendor 和 wiki，等真正做设计时，AI 自动把这些沉睡的知识拉进来参与决策。

手段四：给 AI 套一个「项圈」（渐进实现中）

在 AGENTS.md 之外，我们还在根目录放了一份 `collar.yaml`。**坦率说，这份文件还在后面的 nekoCollar 上渐进落地**，所以这里只讲核心思想，不展开细节。我们把这个项圈抽象成三层，是一套值得记住的心智模型：

- **Identity（身份 / 知识入口）**：告诉 AI「你是谁、去哪读知识」
- **Boundary（边界 / 约束）**：明确 AI「能碰什么、不能碰什么」。核心逻辑是**默认拒绝（deny-by-default）**
- **Validation（验证）**：改完代码必须过的门禁（lint / typecheck）。

一句话：**Identity 管知道什么、Boundary 管能改什么、Validation 管改得对不对**。即便 collar.yaml 本身还在完善，这套三层模型已经通过 AGENTS.md、pre-commit hook、以及后面解法四的软硬双锁在各处启用了。

小结 - 超级个体篇

- 通过入口地图 + collar 三层约束 + 知识库六模块 + 六个 Skill，把整个全栈项目结构化地“装进 AI 的脑子”。
- 一套完整且具纵深的知识生命周期，让 AI 从第一天起就“懂”这个项目。也为后面加人提速打下了地基。

加人提速期：基于能力分发，
横向复刻出「超级研发小分队」

问题：如何把超级个体效率复刻到整个研发小队上

单人攻坚了一个月后，进入第二阶段，项目要加人了。于是核心问题变成：**如何把单人的效率复刻到多人协同上**？多人协同**最怕两件事**：一是**冲突**（大家改到同一处），二是**上下文断裂**（为了工程规范把逻辑外置，结果 AI 看不懂了）。

手段一：借鉴 Web2.0 的站点地图，物理隔离分工

熟悉 Web2.0 早期的同学会记得「站点地图（sitemap）」这个东西。一个网站上线前，先画出它有哪些分区、每个分区有哪些页面，然后不同人认领不同页面并行开发。站点地图的价值在于：**在动工之前，就把整个系统的疆域横向铺开、切分清楚。**

我们把这个古典思路搬到了 AI 研发时代。`docs/specs/` 就是 MDI 的站点地图，它按业务地图横向铺开：

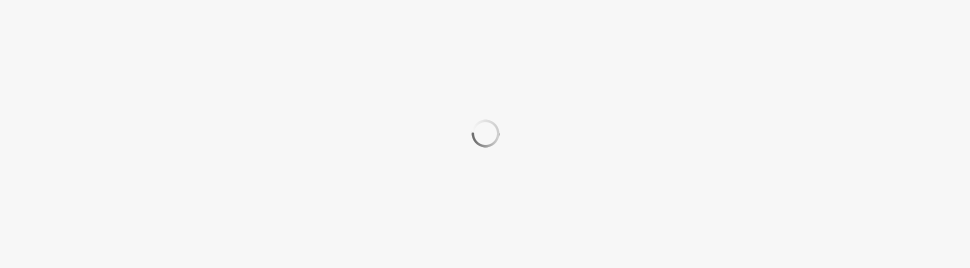
```
docs/specs/
├── 00_[站点设计]/          ← 站点整体骨架
├── 01_[业务地图]管理端/     ← 16 个功能模块（最重）
├── 02_[业务地图]用户端/     ← 6 个功能模块
├── 03_[业务地图]小二端/     ← 14 个功能模块
├── 04_[业务地图]专家端/     ← 1 个功能模块
└── 05_[业务地图]评测端/     ← 2 个功能模块
```

一个业务地图 = 站点的一个分区，一个功能模块 = 分区里的一个页面/功能区。

数字前缀保证目录天然有序。这就是「基于站点地图思路横向铺开」的直接映射。另外，我们在相关同学进入开发前，一开始就确认清楚其负责的产品前台路由、后台 API 领域、负责的产品模块、负责的 spec 范围。任何边界之外的协同都要和项目组对焦。

手段二：四种变更类型满足所有需求演进行为

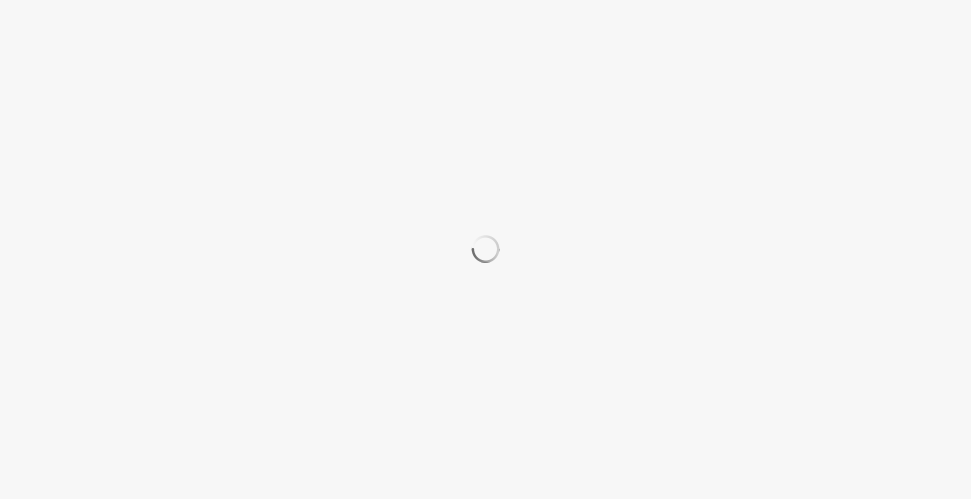
站点地图铺开后，具体到每一次变更，我们用四种类型的需求迭代方式来组织：



一个 feature 的三级结构长这样（以「管理端/创建项目」为例）：

```
01_[业务地图]管理端/
└── 01_[功能模块]创建访谈/                    ← feature
    ├── raw提示词.md                          ← 原始需求（只追加）
    ├── [技术方案]管理端-创建访谈.md           ← 主 spec
    └── patches/                              ← 挂在 feature 上的 patch
        ├── PATCH-001_TPP同步代理接入/
        ├── PATCH-002_TPP异步代理接入/
        └── ... PATCH-011（共 11 个 patch）
```

这里藏着一个层次差异：**feature / patch / sunset 都活在 `docs/specs/`（正式规格层），而 blueprint 刻意放在 `docs/wiki/blue-print/`（自由探索层）**。因为蓝图是“还没确认要如何沉淀成 spec 前的基础能力构思”，它的命名前缀体现「成熟度阶梯」：`[调研]` → `[讨论稿]` → `[技术方案]...V4-MVP`。当调研成熟，才“毕业”迁入 specs 成为正式 feature。



一个功能的完整生命周期 —— 从蓝图预研到日落存档

为什么这样切分能让人天然分开？ 三个层面：

1. **目录即认领单元**：A 认领「创建项目」、B 认领「项目循环」，各自的 spec 目录物理隔离，冲突可能性降到最低。
2. **patch 让并行不打架**：小改动走 PATCH 补丁，多人可以在同一 feature 下并行提交 patch，主 spec 只在稳定后收敛。
3. **生命周期明确**：大前期设计走蓝图，功能下线走日落，稳定功能就走 feature + patch

- 关键规格一：patch 的双向指针解决“主文档与补丁”的语义二义性

patch 最值得展示的不是模板，而是 **双向指针（bidirectional pointer）**机制。当一个 patch 要替代主方案里的某段内容时，如果只在 patch 里说“我改了 X”，读主文档的人不知道；如果只在主文档划掉“X”，读 patch 的人不知道来龙去脉。我们强制双向建立指针：

```
# ① Patch → 主文档（Patch 开头声明覆盖范围）
## 覆盖范围
本 Patch **替代**主方案 `[技术方案]管理端-创建项目.md` 以下内容：
| 被替代段落 | 原方案 | 新方案 |
|---|---|---|
| POST /api/project/{id}/refine-goals | SSE 流式，调 DashScope | 废弃，改
```

```
# ② 主文档 → Patch（被替代段落加删除线 + superseded 指针）
> **已被 [PATCH-001_TPP同步代理接入](./patches/PATCH-001_TPP同步代理接入/)
~~点击后调用 POST /api/project/{id}/refine-goals，SSE 流式展示~~
```

我们会强制要求 **Patch 自身必须可独立阅读**，不依赖读者先读主文档。这对 AI 尤其友好，其从任何文件切入都能拼出当前生效的真相，不被过期描述误导。

- 关键规格二：sunset 实现旧功能优雅日落

旧功能下线时，如果直接 `git rm`，那段历史上下文就永久丢失了，未来排查灰度期的线上问题会抓瞎。我们用 sunset（日落）机制让旧 spec 优雅退场。下面是真实案例，用户核心循环从 V1 自建 LLM Agent 迁移到算法团队的 TPP 服务：

```
# [技术方案] 用户端 – 核心循环
> ⚠️ **日落声明（V1 范式 · 自建 LLM Agent）**
> 自 2026-05-26 起，用户核心链路迁移至算法团队 TPP 黑盒服务。本文档进入凝固期：
> – ❌ 不再修订章节内容（保留 T0 时刻的纯粹历史）
> – ❌ 不再新增 PATCH（V1 时代 PATCH-001~006 已封存）
> – ✅ 仍可阅读用于排查灰度期 V1 范式的线上问题
> 迁移决策：ADR-010
> 预计清场时间：V2 全量切换且稳定 1 周后，本文档及关联 V1 文件将被 git rm。
```

上面这个模块是全项目最佳的主线案例，它**同时经历了四种类型**：V4 全双工方案还在预研（blueprint）、V2 作为正式 feature、V1 时代累积了 PATCH-001~006（patch）、V1 整体日落（sunset）。一个模块把四种类型走了个遍。核心理念就是，**sunset 在我们这里不是「把文件删掉」，而是一份带状态机的可执行迁移剧本**。

手段三：提交时，代码到 spec 的反向提醒

「站点地图」把疆域切好了，但代码写着写着可能会偏离 spec。我们有专门的 skill 在**提交代码时做代码到原始 spec 的检查**。这里的设计哲学非常关键：

> 「Spec 是意图源，比代码更早期。代码偏离 Spec 不一定是错误——可能是编码过程中发现了更好的方案。」

所以提交时**不自动反向更新 Spec**，而是输出差异清单，把决策权交给人：

```
### Git 提交触发：Spec-Code 差异检测（需求澄清）
| # | 差异点 | Spec 描述 | 代码实现 | 建议 |
|---|---|---|---|---|
| 1 | 字段类型 | status: String | status: Integer（枚举值） | 反向更新 Spec
用户逐条决策：
  ✅ 反向更新 Spec（以代码为准）
  ❌ 保持 Spec 不变（代码后续修正回来，记 TODO）
  ⏭️ 暂时搁置
```

这把「文档变化」从静默忽略变成了显式决策。配合工程侧的 lint 质量门禁和 collar-changelog 强制约束，构成「Agent 语义检查 + git hook 质量检查」的双保险。

手段四：上下文缝补

现在讲这个阶段我认为 最有价值 的部分。

多人协作到一定阶段，一定会为了发布效率，遵循工程范式把某些硬编码（比如 prompt 逻辑）迁移到 Diamond、DB 或其他持久化介质上。这在传统研发时代是标准的“最佳实践”，配置与代码分离嘛。

但在 AI 研发时代，这是一个隐蔽的陷阱。

想象一下：原本 Agent 的行为逻辑（提示词）写在代码里，codeAgent 一读代码就知道这个 Agent 会怎么干。现在你把提示词挪到了 DB，代码里只剩一个 `promptService.load(agentCode)` 的调用框架。codeAgent 再读代码，就完全不知道这个 Agent 的具体行为了——它会在关键决策上误判，或者给出那句让人血压升高的话：

> 「但我不知道这个 Agent 的具体行为，请你去对应平台修改相关的提示词。」

这就是**推理链断裂**。你以为做了工程优化，实际上把 AI 的上下文捅了个窟窿，研发效率断崖式下跌。

我们 MDI 项目组内由此立下一条原则：

> 任何导致上下文异常的研发架构变动，都必须用适合的方式缝补缺损的上下文。

这背后是一个更根本的认知 —— **AI 时代，human in the loop 越少效率越高，所以 AI 上下文始终是效率的关键**（注意：这里说的是效率，不是质量）。DB 相关的 MCP 为什么好用？就是让业务逻辑链路重新完整、让 codeAgent 上下文齐全。

从这个视角看，DB 等外接的查询类研发 MCP 的价值就是把“外置到 DB 的逻辑”重新拉回 AI 的视野。

下面是我们缝补的三个真实案例。

- 案例一：把外置的动态提示词/skill 拉回 AI 视野（MCP + 只读查询）

MDI 的多 Agent 报告生成管线，其业务逻辑（prompt 内容）不在代码中硬编码，而是通过 **Prompt 工作台动态管理、独立发布**。代码里只有调用框架和数据流转——典型的“上下文缺损”场景。

我们的缝补方式是提供一个 MDI 专属的 MCP Server，暴露 20 个只读查询工具，其中第 7 组「Prompt 工程」专门做 “**根据原本的锚点，查询当前实际生效的动态提示词/skill 内容**”：

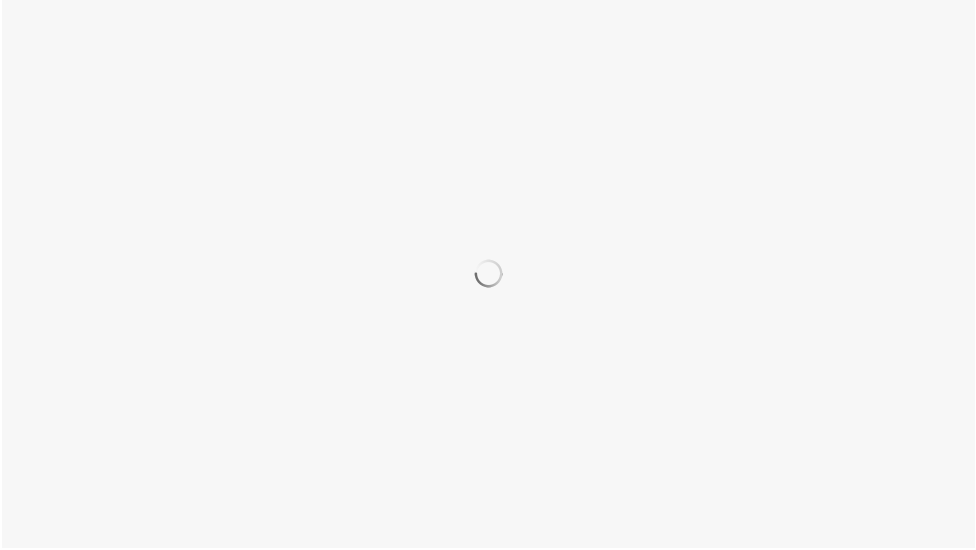
```
# mdi-query MCP —— Prompt 工程组（缝补核心）
query_prompt_manifest(sceneCode, env?)    # 锚点=场景码+Agent码，返回每个 A
query_prompt_content(id)                   # 用 promptId 拿到完整 Mustac
query_prompt_versions(sceneCode, agentCode)  # 版本迭代历史
query_prompt_intermediates(insightId)      # 每个 Agent 的实际输入/输出/耗
```

codeAgent 任何时候都能通过 MCP 取到“当前实际生效”的 prompt，那句“请你去平台自己改”的推理链断裂就不会再出现。

但光有 MCP 还不够——工具“能查”，不等于 AI“知道该查”。所以我们额外配了一个 skill，它就是这个 MCP 的**使用说明书**，专门教 codeAgent 如何主动地把自己的上下文补全。它明确回答三个问题：

- **何时查？**：skill 里定义了触发条件，涉及到相关上下文会主动调 MCP，而不是等人提醒。
- **怎么查？**：固定四步调用链，锚点就是场景码 + Agent 码。
- **查到后怎么用？**：补充 mcp 数据含义，赋能 codeAgent

这才是完整的缝补闭环：MCP 提供“只读可达”的通道（能力层），skill 提供“何时/如何取用”的协议（认知层）。两者合起来，codeAgent 才会**主动地、按正确锚点地**把外置到 DB 的上下文拉回视野，而不是被动等着能力在那里却想不起来用。



上下文缝补的核心 —— 用 SKILL/MCP 把外置到 DB 的动态提示词/skill 按锚点拉回 AI 视野

- 案例二：预发/线上环境差异在 AGENTS.md 里显式声明

我们的预发与线上环境差异对 AI 是个“隐藏地形”——它排查预发问题时，可能遇到无法解释的现象（明明代码对的，就是不生效）。

缝补方式：在 `AGENTS.md` 的「关键约定速查」里**显式告知** codeAgent 存在环境差异、遇到不可解释的问题要优先考虑环境，并把细节沉淀成 ADR。

- 案例三：Transform 脚本用声明式注释保留上下文锚点

我们把一些运营逻辑外置到 Transform 平台（Groovy 脚本），但用一套 `//!` 声明式注释把上下文锚点保留在代码里——在 Groovy 眼里是普通注释，对平台却是元数据，**代码即锚点**，这样即便逻辑外置到了平台，AI 读脚本源码仍能从这些声明里知道：这个脚本叫什么、风险几何、参数是什么、能在哪个环境跑。这依然是“外置逻辑但保留可解释性”的同一种缝补哲学。

小结 - 超级研发分队篇

- 横向铺开：基于一套站点地图、四种变更模式，让多人协作从一开始就自然分开、冲突面最小
- 上下文缝补机制确保工程优化不以牺牲 AI 上下文为代价。任何导致上下文异常的架构变动，都要缝补回来。

高速迭代期：引入「技术哨兵机」从更高维维稳

问题：高速迭代期如何确保研发小分队质量不劣化？

进入第三阶段，每个人的效率都上来了，节奏变成每周一个大版本、预发同时十几个分支。新问题浮现：**效率提升后，人与人之间对新输入的上下文交换效率成了瓶颈**。这就会出现这样的情况：我上午改了后端接口签名，你下午改了前端调用 —— **各自在本地都跑得通，合到预发业务逻辑一冲突，甚至可能要延续很久之后才炸，甚至会酿成线上问题**。

这种「隐形冲突」在多人高速迭代下会指数级增长，是 **AI 研发质量劣化的头号杀手**。我们必须引入新的上下文交换机制。

手段一（一次失败的尝试）：Agent Manage（编码态监控）

我们最初的想法很直觉：既然要同步上下文，那就**直接读取大家本地编码阶段的输入和 Agent 输出**，希望在编码阶段就把冲突扼杀。这就是我们做的 `nekocollar-app`（Agent Manage 方式）。

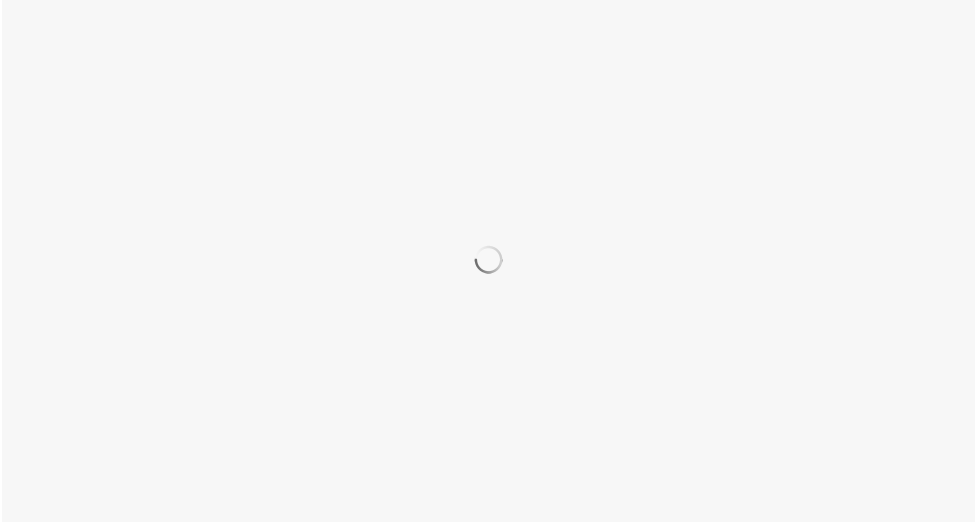
所谓的 nekocollar-app，就是一个全局的 codeAgent 信息采集器，他可以监听 cursor、qoder、claude code 这些 agent 进程的输入输出，实现 AI Coding 信息的全量获取并管理（这个无论是开源还是我司内部都有很多实践了）。

我们是希望所有同学都转一个本地 app，然后监听所有通过 collar 这一套模式运行的项目，从其中采集到我们需要的项目协同信息并实现冲突的提前判别。

但实际跑下来，我们遇到了两个致命问题，**这一块是失败的**，如实记录如下：

1. **噪音太多、信噪比极低**：大家在没有 release 自己代码前，本地编码态充满了不成熟的探索——写了又删、试了又改、跟 AI 反复对话调整。这些“过程态”绝大部分是无用噪音。
2. **成本过高**：要监控每个人本地的编码态输入输出，token 消耗大得惊人，各 Agent 的行为监控也很难做（每个人用的工具、习惯都不一样）。

失败的根因，事后复盘很清楚：**我们把介入时机放错了**。编码态是“意图尚未定型”的阶段，此时的信息熵最高、确定性最低，用它做冲突检测成本过高。



失败与成功的分野 —— 介入时机从“编码态”后移到“预发集成态”。

▌手段二（这次有戏）：把触发节点放在预发检测上，还原一个“谁的开发机都没有的”最终运行环境

复盘之后，我们把触发节点从“编码态”后移到“预发集成态”。这就轮到 `collar-daemon` 这个项目登场了。它的定位很纯粹：一个**跑在独立机器（目标形态：一台 Mac mini）上长驻的守护进程**，本质是“团队研发质量的哨兵”。技术栈极简——**原生 Node.js，零第三方依赖**，只用 `child_process` / `https` / `net` / `fs` 等内置模块，部署到 Mac mini 连 `npm install` 都省了。（极简的原因有两点，一个是我需要快速实现我要的功能，二是实现了我要的功能并稳定运行后，要考虑部署到云端）

它做三件事，是一个闭环：

1. **代码环境还原**：持续把「线上」和「预发」两套工作区的所有子仓库同步到各环境的最终态。
2. **业务逻辑冲突检测**：预发一拉到新提交，就把“谁改了什么”喂给云端 AI Agent 做隐形冲突分析。
3. **告警闭环**：Agent 发现冲突，通过钉钉机器人 + AI 表格告警到群里。

团队里每个开发者本地只有**自己那一部分**的改动，分支还各不相同。所以一直以来，互联网项目多人协同时，除了集成环境本身，**没有任何一台开发机拥有“所有人的最新提交合在一起”的完整环境。** 但恰恰是这个“合在一起”的环境，才是冲突真正暴露的地方。

我们的 collar-daemon 在一台独立机器上，实现了持续把两套工作区拉到最终态的上下文构建工作：

```
// src/config.js —— 双环境，各自独立轮询间隔（预发更快）
workspaces: [
  { env: 'prod', path: '.../workspace-mdi',     interval: 600 },  // 
  { env: 'pre',  path: '.../workspace-pre-mdi',  interval: 300 },  //
],
```

关键在预发的处理——**每个子仓各自独立发现自己最新的 release 分支**，而不是统一切一个分支：

```
// src/sync.js —— 按 committerdate 倒序自动发现最新 release 分支
async function discoverReleaseBranch(repoDir) {
  const refs = await git(repoDir, [
    'for-each-ref', '--sort=-committerdate',   // ★ 按提交时间倒序
    '--format=%(refname:short)', 'refs/remotes/origin/',
  ]);
  const combinedRegex = new RegExp(   // 前端仓 def_releases_* / 后端仓
    `^origin/(?:${CONFIG.releasePatterns.map(globToRegex).join('|')})`)
  const match = lines.find((l) => combinedRegex.test(l));
  return match ? match.replace(/^origin\//, '') : CONFIG.prodBranch;
}
```

于是：前端仓走 `def_releases_2026xxxx`、后端仓走 `releases/2026xxxx`，**各服务当前要上预发的那版代码，被物理地聚合到同一台机器的同一个工作区里**。这不是靠 git merge 造一个临时分支，而是在工作区层面**模拟出了“预发合并态”**。一个可运行、可被 AI 通读的完整快照。因为是持续轮询（预发每 300s），这份“最终态镜像”始终跟着团队提交实时更新。

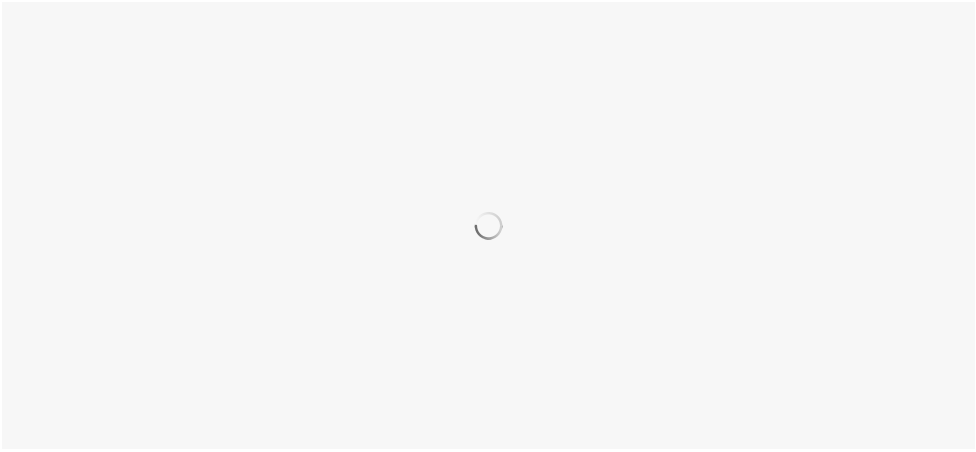
而且它不需要人工告诉它“这周的预发分支叫什么”，它会 **自动切换到最新预发态**。

为什么“始终维护一份 release 集成分支”如此关键？ 这一点你看以下例子就明白了：

- A 改了某个接口的签名，B 恰好在调用这个接口——两人各自的分支都编译通过、自测通过，只有当两份代码合到一起，签名不匹配的问题才暴露。
- 甲改了共享 Prompt 的槽位 key，乙依赖旧 key 读取——各自分支都对，合并后线上就静默读不到、悄悄失效。
- DB 字段变更、并发时序、共享配置……这类冲突**在任何单一开发分支上都“一定不存在”，必须等集成后才现形**。
- …

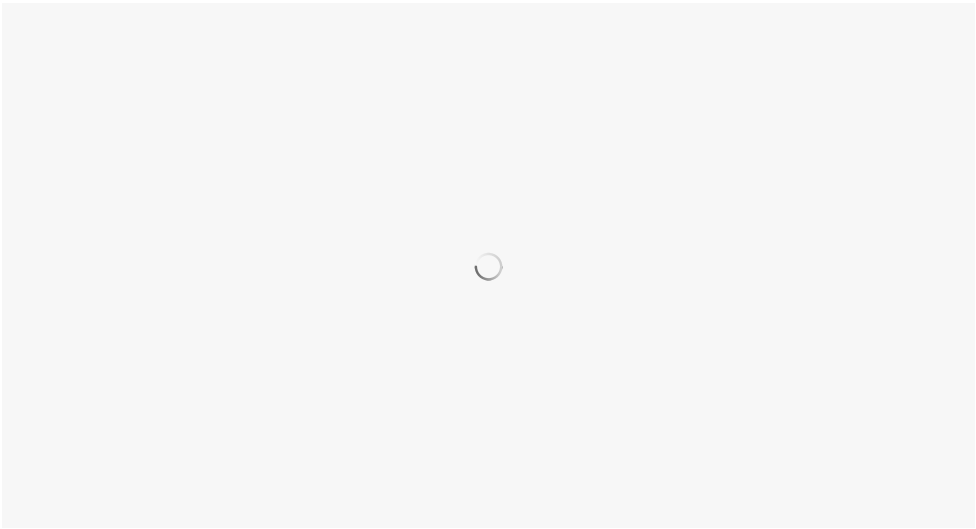
只有在这份“所有人最新提交合在一起”的集成代码上做查询和检测，才能命中那些藏在合并处的真实问题。release 集成分支是“冲突唯一会现形的地方”，哨兵机就是 24 小时盯着这个地方的人。

这不是纸上推演。下面几条，都是哨兵机在集成态上真实播报出来、且**在各自开发分支里根本查不出问题**的例子，其中会有多种类型的报错：



这是数字人一天 24 小时内发现的问题

哨兵机不是每次都报警，只在真有合并风险时才 @ 责任到人，这正是它信噪比能维持在八成以上的原因。



collar-daemon 的核心价值 —— 在一台哨兵机上还原出全团队的预发合并态

信息采集时绝不阻塞和破坏本地工作区。我们会精确提取每个新提交的作者、message、变更文件，组装给 AI 的上下文时，按仓库分组、每条突出提交人名字——因为多人协同排查冲突时，“谁改的”是第一线索。

手段三（深挖）：从 daemon 发现，到 QoderWake 处理，再到 钉钉 留痕

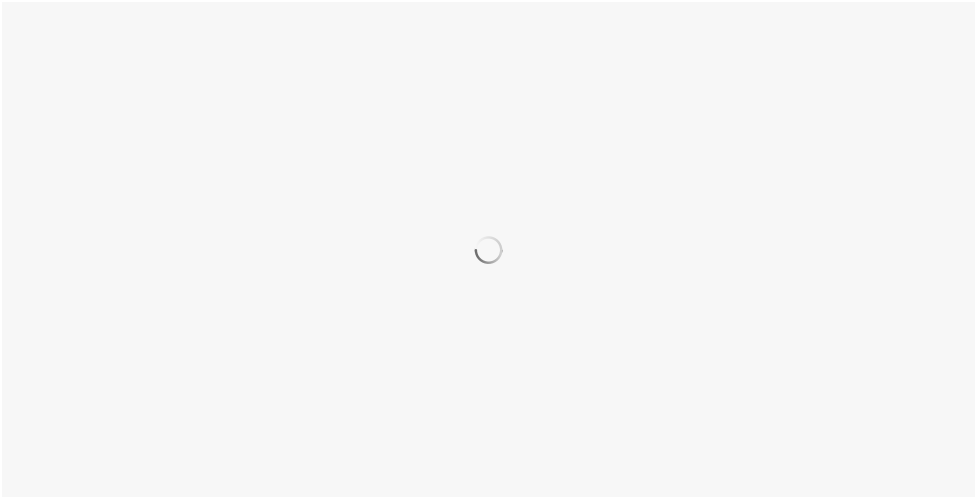
daemon 代码不做任何规则判断、不内联任何 prompt。它只把结构化上下文 POST 给云端的 QoderWake Automation Agent，来判定冲突。

易变的智能（prompt 编排）放在云端 Agent，本地进程可以做到零依赖、长期稳定不用改。

QoderWake Agent 拿到“提交上下文 + 完整代码”后综合分析，命中冲突就 **群告警 + AI 表格写入**。相关同学在钉钉里直接看到“谁的哪个提交与谁冲突”。日常跨研发同学的代码问题排查、trace 检查、版本 changelog 的业务逻辑冲突，都由这同一条 Agent 分析链路的不同产出维度承载，闭环起来。

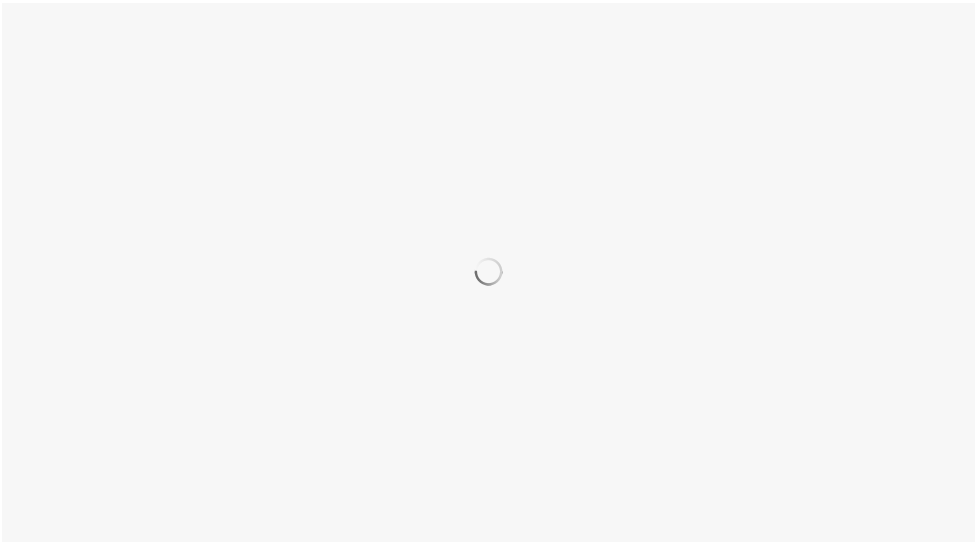
> 还有个工程小巧思：QoderWake 的管理界面 daemon 硬编码绑定 `127.0.0.1`，团队无法远程访问。collar-daemon 用 `net.Server` + `pipe()` 做了一个**同端口不同 IP 的 TCP 透明转发**，把 `127.0.0.1:19820` 暴露到内网 IP，绕开了无法改绑的限制，让全团队都能远程看这台哨兵机的界面：

```
// src/port-forward.js —— 同端口不同 IP 共存，双向 pipe
const server = net.createServer((clientSocket) => {
  const upstream = net.connect(cfg.targetPort, cfg.targetHost);  // →
  clientSocket.pipe(upstream);
  upstream.pipe(clientSocket);       // 双向转发，支持并发多连接
});
server.listen(cfg.listenPort, lanIp);   // 绑定内网 IP，与 127.0.0.1 不
```



当然，如果冲突检测只停留在「群里发条告警」，价值会大打折扣。告警会被刷走、没人跟进、无法度量。所以我们把 Agent 每轮产出的冲突**沉淀成一张钉钉 AI 表格**，并在其上做**状态流转**管理，让「发现问题」进一步变成「闭环问题 + 可度量的研发质量」。

- **每条冲突都是一条可流转的记录：**相关同学看到后，把状态从待回复流转为**「问题存在」/「误报」/「已修复」**中的一个。这样既闭环了问题，也反过来校准了 Agent 的准确率。
- **北极星指标设定：**我们哨兵机北极星指标是 **周冲突有效发现率**（即 Agent 报出的冲突里有多少被人工确认为真问题）。



冲突沉淀为 AI 表格并按周流转，有效发现率稳定 > 70%，成了团队离不开的基础设施

跑出来的真实成效。 这套机制上线运营一段时间后，从趋势看已经很能打：

- **业务冲突有效发现率稳定超过 70%**——也就是 Agent 报出的冲突，七成以上都是真问题、不是噪音。相较于之前的本地检测方案，**信噪比直接从“不可用”翻到“八成命中”**。
- **每周能发现 40+ 个实实在在的多人研发潜在问题**，这些都是分散在各自开发机上、单人视角根本看不见的隐形业务埋雷。
- 更有意思的是团队心态的变化：**现在项目组里如果哪天这个“禾小荞机器人”没有发冲突检测，大家会普遍觉得心理不安**——总担心是不是哪里悄悄埋了雷没被发现。团队会对一个 AI 工具产生这种“没有它就不踏实”的依赖，说明它真正嵌进了研发流程。

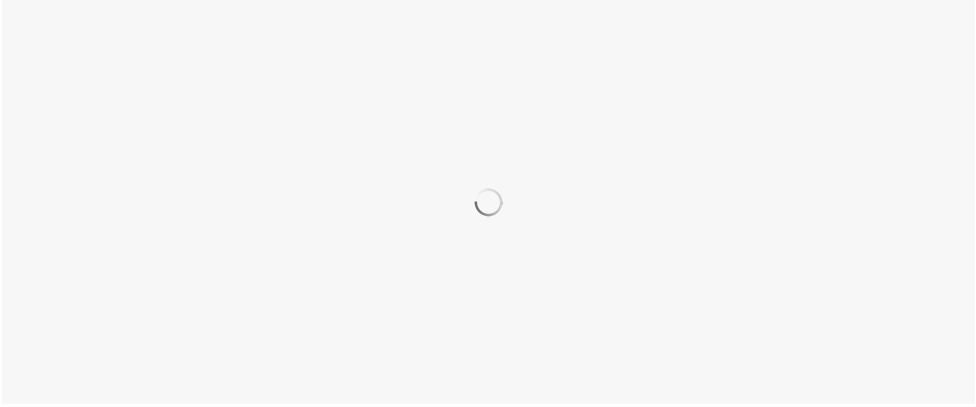
小结 - 高维哨兵机器人篇

- 我们用一台零依赖的哨兵机，还原出“谁的开发机都没有的”预发合并态镜像，把每轮新提交的隐形冲突交给云端 Agent 检测
- 有效发现率稳定 >70%、每周揪出 40+ 个真实的多人研发潜在问题。项目组已经重度依赖起了它。

稳定运作期：为「整个项目组的成员」带来效率和稳定性的平权

问题：研发的效率和质量提升了，那项目组里的非研发同学呢？

项目稳定运作后，我们意识到一个更大的机会：**能不能把这份效率和稳定性，同步交付给项目组里的非研发同学？**测试、产品、设计——他们同样需要理解代码、需要基于当前系统提出改动、需要一个“所见即所得”的编辑体验。



互联网各角色之所以都需要围绕代码上下文协作，是因为**代码是距离用户真实体验最近的一份“可执行事实”**。

产品文档描述希望发生什么，设计稿描述用户应该看到什么，测试用例描述系统应该满足什么。这些上游产物本质上都是最终系统的不同投影，在传递过程中必然受到技术约束、接口实现、历史兼容和运行环境的影响。用户最终体验的是代码与配置实际运行出来的行为

> *所有互联网工种，多年来其实一直是围绕“代码”这一互联网模式的最终产物，跟着项目一起进行中心化协同的。*

如果协作始终停留在上游产物之间，信息差就会沿着交付链不断累积。只有让产品、设计、测试的判断持续回到代码与运行态上校准，才能尽早发现“产品以为已经实现、设计以为能够还原、测试以为覆盖完整”与真实系统之间的偏差。更准确地说，用户收到的是系统行为，而代码上下文是产生这一行为之前，最后一份可版本化、可追踪、可验证的共同事实。**围绕代码协作，不是让所有人进入 Git，而是让所有角色都基于最终可执行事实工作**。

但直接把 git 权限、把 AI 编码能力交给非研发角色，风险极高：一个误操作就可能污染代码库。

于是我们正在开发 `NekoCollar`，一个**计划将测试、产品、设计等角色融到一起、共享研发效率 + 稳定性的云端上下文协作系统**。它是 collar SDD 体系的“平台化”兑现（还记得解法一里 `collar.yaml` 那些“平台扩展占位”字段吗？在这里它们开始生效了）。

这一步是我们认为非研发人员参与“代码开发”的“终局形态”。

前三章的手段，解决的都是“研发同学之间”的协同，参与者都懂代码、懂 git、有工程素养，约束可以相对“软”（靠文档、靠规范、靠自觉）。但当参与者扩展到测试、产品、设计——他们不必懂代码，却要能安全地基于代码库表达意图——约束就必须从“软”变“硬”。因为你不能假设一个不懂 git 的产品经理会记得“不要 push 到主干”。

参与者的工程素养越低，系统的约束就必须越硬。 这是本章与前三章最本质的区别。

NekoCollar 的设计哲学可以一句话概括：**在效率上做加法，在稳定性上做物理约束**。下面分两条线讲。

设计思想一：效率线，让非研发角色也能“上下文编排式协同”

- **云端沙箱一键开工，物化上下文。** 非研发同学不需要在本地配环境。系统用一套异步编排（`WorkspaceProvisionOrchestrator`，`@Async` 五步流水线）在云端 Aone 沙箱里一键拉起工作环境：

```
Step1 CreateInstance（同步，前端立即拿 instanceId）
Step2 CREATING_SANDBOX（Aone SDK 起沙箱）
```

```
Step2.5 INSTALLING_CLI（装 claude-cli）
Step3 PrepareWorkdir（★ 先 clone 知识库作外层目录，再把代码仓 clone 进知识库
Step3.5 INJECTING_HISTORY（reprovision 时恢复对话记忆）
Step4 STARTING_BRIDGE（启动 Bridge + Claude Agent）
Step5 CONNECTING_AGENT（markActive → RUNNING）
```

注意 Step3，**先 clone 知识库作为外层目录，再把代码仓 clone 进知识库内部**——这就是目前我们研发态实际用到的真实“上下文”环境。

- **所见即所得的产品编辑体验（这是我们和其他全栈研发平台最不一样的一点）。**

市面上大多数「AI 改前端」的产品，要么让你在一个隔离的沙盒预览页里看效果，要么给你一段代码 diff 让你自己想象。**这两种都不是“真实页面”**——产品、设计同学看到的，始终是一个“仿真环境”，和线上真正跑的那个页面隔着一层。

我们花了不少力气做的，是让非研发同学 **直接在预发/线上的真实页面上，看到沙箱里 AI 实时编辑的效果**。这套机制的完整链路是这样的：

1. 沙箱内跑真实的 dev server。 工作空间开机时，对配置了 `devServerPort` 的前端仓库，在 Aone 沙箱里后台拉起一个真实的开发服务器 —— 就是我们本地开发天天用的那个 `npm run start`：

```java
// WorkspaceProvisionOrchestrator.java —— 沙箱内后台启动真实 dev server
String cmd = String.format(
  "lsof -ti:%d | xargs -r kill 2>/dev/null; "                // 先清端口
  + "cd '%s' && nohup sh -c '"
  + "npm install --registry=https://registry.npmmirror.com 2>&1"
  + " && npm run start -- --host 0.0.0.0 --port %d 2>&1'"    // 真实 n
  + " > /tmp/dev-%s.log 2>&1 & echo 'DEV_SERVER_BG_OK'",
  port, repoDir, port, safeRepoName);
```

2. 把 dev server 端口暴露并注册到 NekoCollar。 通过 Aone Sandbox SDK 把沙箱内部端口映射成一个外部可访问的 URL，再把数据写进工作空间实例的数据中，完成“注册”。平台会根据结果专门提供一个给浏览器插件消费的接口：

```java
// CollarWorkspaceController.java —— 供浏览器插件拉取端口映射
@GetMapping("/{workspaceId}/dev-servers")   // 返回 [{contextUri, repo
```

```
public NovaResult<List<DevServerVO>> devServers(@PathVariable Long wo
// Service 里不是读缓存，而是实时重新 getEndpoint，保证返回最新可用端点
```

3. 浏览器插件（`nekocollar-extension`）自动代理。 这是最关键的一环。当你在浏览器打开预发/线上真实页面时，插件自动从 NekoCollar 后端拉取当前工作空间的 dev server 映射，然后动态注入重定向规则，**把该页面从 CDN（`g.alicdn.com` / `dev.g.alicdn.com`）加载的前端资源，实时重定向到沙箱 dev server 的 endpoint**。插件只在 NekoCollar 页面生效、规则严格限定到当前标签页（不污染其他页面），并带 20 秒 TTL 缓存 + 轮询刷新，保证 endpoint 变化能及时同步。

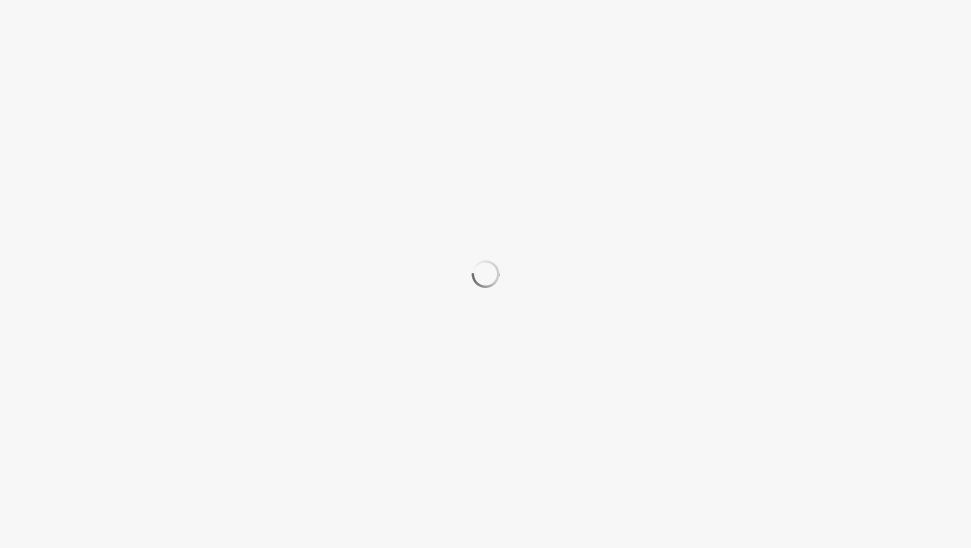
```
// nekocollar-extension/background.js —— 把 CDN 资源重定向到沙箱 dev ser
addRules.push({
  id: ruleId++,
  priority: 1,
  action: { type: 'redirect', redirect: { regexSubstitution: `${endpo
  condition: {
    // 匹配"这个仓库"在 CDN 上的资源路径，捕获资源真实相对路径 (.+)
    regexFilter: `^https?://dev\\.g\\.alicdn\\.com/${escaped}/[^/]+/(
    resourceTypes: ['script', 'stylesheet', 'image', 'font', 'xmlhttp
  },
});
```

4. 闭环：所见即所得。 于是完整的体验就串起来了。产品/设计同学在沙箱里跟 codeAgent 对话改页面 → dev server 热更新产物 → 浏览器插件把真实页面的资源代理到沙箱 → 用户在预发/线上的真实页面上，即时看到 AI 编辑的结果。不是仿真、不是预览、不是 diff，就是那个真实页面本身，实时变化。和我们研发在本地开发全栈应用的体验是一模一样的！

这就是我们和其他平台不同的地方之一：**原本只有研发用本地 whistle/charles 代理才能做的“调试真实页面”，现在通过沙箱 dev server + 浏览器插件自动代理，无门槛地交给了非研发同学**。产品经理不用装本地环境、不用懂代理配置，打开真实页面就能看到改动效果。

那服务端的改动怎么生效？ 前端资源可以靠 dev server 热更新 + 插件代理即时看到，但如果这次改动还牵涉到后端接口（比如 codeAgent 顺手调整了某个接口的返回结构），就没法用同样的方式立刻生效——**服务端改动要真正上预发/线上，受限于发布流程的时间成本和部署复杂度**（走一遍构建、审批、部署往往是分钟级甚至更久）。对产品/设计同学的“边改边看”体验来说，这个等待是致命的

所以这里我们规划的取舍是：服务端的改动同样交给浏览器插件用 mock 的方式就地拦截、直接返回改后的响应，而不是等它真的发布。这样产品同学改完接口逻辑，页面上立刻能看到对应的数据变化，闭环依然是“所见即所得”的。之所以敢这么做，是因为非研发场景下对业务后端逻辑的改动，影响面是可控的。（需要说明：当前插件已扎实落地的是前端静态资源代理这条链路；服务端接口 mock 属于同一思路下的规划中环节）



非研发同学直接在预发/线上真实页面上对话式修改

设计思想二：稳定性线，软硬双锁把危险操作变成“物理不可能”

这是 NekoCollar harness 层设计的部分。

- **硬约束（协议层，不可绕过）：clone 后立即作废 push URL。** 沙箱 clone 完代码后，第一件事就是遍历所有 git 仓库，把 push URL 改写成一个不存在的仓库名：

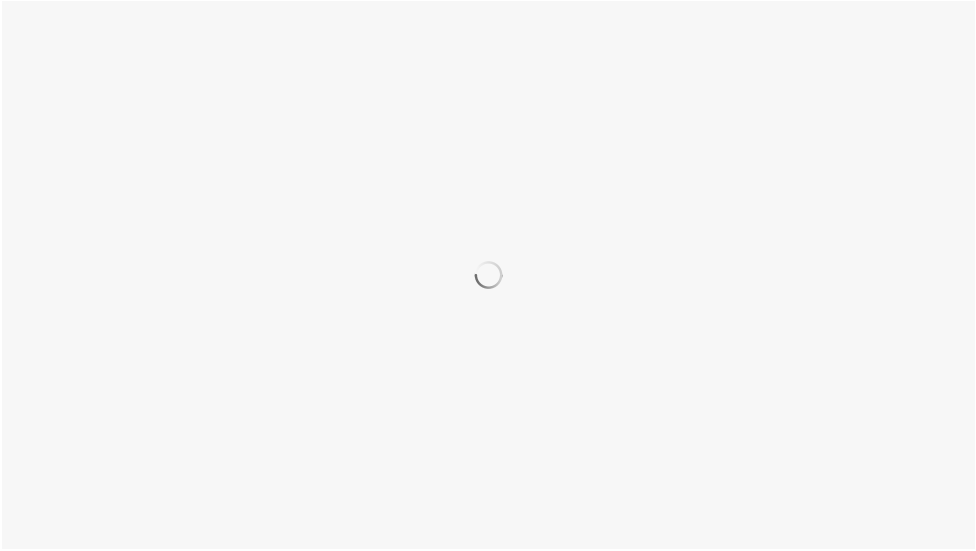
```
// WorkspaceProvisionOrchestrator.java —— disablePushForClonedRepos()
// 把每个仓库的 push URL 改写为 PUSH_DISABLED，任何 git push 在协议层直接失败
String cmd = "find '...' -name .git -type d -maxdepth 3"
  + " -exec sh -c 'cd \"$(dirname \"$1\")\" && git remote set-url --pu
```

此后任何 `git push` 都会报 `fatal: 'PUSH_DISABLED' does not appear to be a git repository`。**沙箱里的代码只能进、不能推出去污染主干。**

- **软约束（行为层，减少无谓尝试）：Claude Code 权限白名单。** 写入沙箱的 `~/.claude/settings.json`，`permissions.allow` 白名单**只放行拉取类 git 命令**

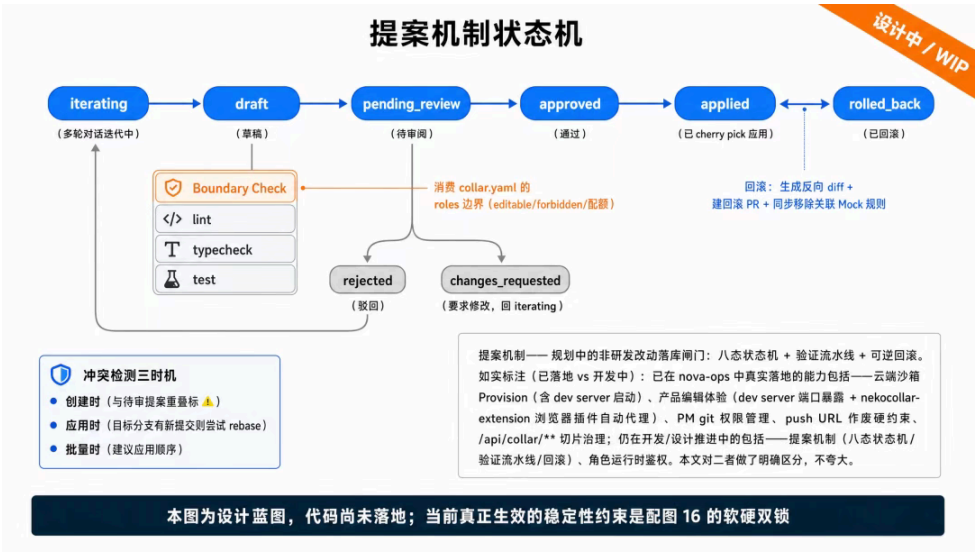
（fetch/pull/merge/rebase/checkout/add/status/log/diff 等），push / remote 不在白名单，在 ACCEPT_EDITS 模式下审批直接失败被拦截。

这两道锁都已真实落地。在此之上，我们还规划了第三道锁——提案审阅兜底（下面详述，目前开发中）。



代码只进不出

- **不改代码、只 cherry pick 验收后净改动的提案机制**。前面两道锁保证了“代码推不出沙箱”，但还有一个问题：非研发同学在沙箱里改得满意后，这些改动怎么安全地落到代码库？我们为此设计了一套「提案（Proposal）」机制。**先讲清楚我们的设计**，它的核心理念是让非研发角色多轮对话产生的改动不直接落库，而是先沉淀为一份结构化提案（已验收净 diff + 需求 + 决策 + 验收证据），经审阅后才被智能 cherry pick 应用。此外，我们还设计了面向不同角色的约束实践，让定制化、约束化的协作成为可能。



小结 - 超级项目组篇：

- 稳定运作期，构建云端开箱即用的研发上下文 NekoCollar，效率靠云端沙箱 + 上下文物化 + **所见即所得的真实页面编辑** + PM 规则治理来实现
- 稳定性上，git 协议层的软硬双锁已落地，而提案机制、角色运行时鉴权仍在开发中。它是 collar SDD 从“本地 CLI 体系”走向“云端多角色平台”的自然延伸，一个我们正在持续建设的云端终局形态。



结语：AI 时代研发协同发展的四条必经之路

回望这四个阶段，如果要提炼四条直接可复用的经验，那就是如下：

1. **单人起步阶段：**
   a. **解法：构建核心全栈上下文**
   b. **原因：其知识库深度将直接决定原子单位研发效率**
2. **多人扩张阶段：**
   a. **解法：构建 spec 变更规则和站点地图**
   b. **原因：其权责划分清晰度直接决定原子单位效率是否能横向扩散**
3. **敏捷迭代阶段：**
   a. **解法：构建比传统研发模式更高维的 AI 监控机制**
   b. **原因：传统维度手段无法让研发质量和研发效率同步提升**
4. **项目稳定阶段：**
   a. **解法：构建更多易用的协同链路 + harness 工程**
   b. **原因：新链路的设计将直接决定是否能将效率和质量辐射至非研发人员**

回到开篇那条主线：这四步走下来，其实还是那句话，我们要 **把 AI 带来的研发能力一层层往外放大：**

个体 → 团队 → 可持续 → 全员

先让一个开发者成为超级个体，再让超级个体组成能协同的超级团队，再让这支团队可持续地高速跑下去，最后把这份能力辐射到非研发的每一个人。

AI 能力普惠后，人心中那些不固步自封、追求和创新的特质才是最宝贵。

07 团队介绍

本文作者卡狸，来自淘天集团-天猫技术团队。该团队是淘天集团的核心技术引擎，支撑国家补贴、百亿补贴、天猫国际、天猫APP、新品孵化、天猫行业等淘天核心业务的全链路技术落地。我们以高并发架构、智能算法和数据驱动能力，为上亿消费者打造行业标杆级购物体验，为百万商家构建高效增长的技术底座。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

964篇原创内容

公众号

阅读 8491

留言

写留言

还没有留言，来写首评